

Neural Network-Based European Option Pricing

under Black-Scholes SDE dynamics

*A supervised learning study of neural networks
as pricing surrogate models*

 [nn-option-pricing](#)

Matteo Giorgi
Lorenzo Ghiotti
Anne Frances Nalumansi
Alice Pedron

Research Question

Main question

Can a feed-forward neural network accurately approximate the Black-Scholes pricing function for European call options?

The project is formulated as a controlled supervised-learning problem:

$$(S_0, K, T, r, \sigma) \longmapsto C_{BS}(S_0, K, T, r, \sigma).$$

Takeaway

Since the target is known exactly, the approximation error can be measured directly.

Why This Problem Is Meaningful

Option pricing often requires evaluating many contracts across many parameter configurations.

- ▶ Closed-form formulas are fast but available only in special cases.
- ▶ Monte Carlo is flexible but can be expensive when repeated many times.
- ▶ A trained neural network can produce prices through a single forward pass.

Takeaway

The neural network is studied as a pricing surrogate: a fast approximation of a known pricing map.

Black-Scholes Dynamics

Under the risk-neutral Black-Scholes model:

$$dS_t = rS_t dt + \sigma S_t dW_t.$$

The exact terminal distribution is:

$$S_T = S_0 \exp \left[\left(r - \frac{1}{2} \sigma^2 \right) T + \sigma \sqrt{T} Z \right], \quad Z \sim \mathcal{N}(0, 1).$$

Takeaway

This exact expression is also useful computationally, because Monte Carlo can sample S_T directly.

European Call Option

A European call option gives the right to buy the underlying asset at strike K at maturity T .

$$\text{payoff} = (S_T - K)^+ = \max(S_T - K, 0).$$

Under risk-neutral pricing:

$$C_0 = e^{-rT} \mathbb{E}^{\mathbb{Q}} [(S_T - K)^+].$$

Takeaway

All methods in the project are evaluated by how well they recover this price in the Black-Scholes setting.

Black-Scholes Formula

The analytical call price is:

$$C_{BS} = S_0 N(d_1) - Ke^{-rT} N(d_2),$$
$$d_1 = \frac{\log(S_0/K) + (r + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}.$$

Takeaway

The formula provides the clean target variable used for supervised learning.

Monte Carlo Benchmark

Monte Carlo approximates the risk-neutral expectation by simulation:

$$\hat{C}_{MC} = e^{-rT} \frac{1}{M} \sum_{i=1}^M (S_T^{(i)} - K)^+.$$

- ▶ It is flexible and widely used.
- ▶ It is stochastic, so estimates fluctuate.
- ▶ The standard error decreases at rate $M^{-1/2}$.

Synthetic Dataset

The dataset is generated by sampling option parameters and computing the analytical Black-Scholes price.

Variable	Meaning	Range
S_0	underlying price	[50, 150]
K	strike price	[50, 150]
T	maturity in years	[0.1, 2]
r	risk-free rate	[0, 0.05]
σ	volatility	[0.1, 0.6]

Why Synthetic Data?

The project does not aim to model observed exchange-traded option quotes.

- ▶ Synthetic labels give a known ground truth: C_{BS} .
- ▶ The experiment is controlled and reproducible.
- ▶ Real option prices include bid-ask spreads, liquidity, dividends, and implied-volatility effects.

Methodological choice

Real market data would change the research question from approximating Black-Scholes to modeling market prices.

Feature Engineering: Moneyiness

The final feature set includes moneyiness:

$$\frac{S_0}{K}.$$

The neural network receives:

$$(S_0, K, T, r, \sigma, S_0/K).$$

- ▶ $S_0/K < 1$: out-of-the-money region.
- ▶ $S_0/K \approx 1$: at-the-money region.
- ▶ $S_0/K > 1$: in-the-money region.

Supervised Learning Setup

Each observation has the form:

$$x_i = (S_0, K, T, r, \sigma, S_0/K)_i, \quad y_i = C_{BS,i}.$$

The model learns a function:

$$f_{\theta}(x_i) \approx y_i.$$

Inputs and targets are standardized during training, while metrics are reported in original price units.

Neural Network Architecture

The selected model is a feed-forward multilayer perceptron.

Component	Final choice
Input features	$S_0, K, T, r, \sigma, S_0/K$
Hidden layers	fully connected MLP
Activation	SiLU
Output	one scalar price

Takeaway

The architecture is simple but appropriate for smooth tabular regression.

Training and Metrics

The training procedure uses:

- ▶ 70% / 15% / 15% train-validation-test split;
- ▶ MSE loss;
- ▶ Adam optimizer;
- ▶ early stopping on validation loss.

Main metrics:

MAE, RMSE, R^2 , MAPE for prices > 1 .

Software Architecture

The code is organized as a reusable Python package plus command-line experiment scripts.

Module	Responsibility
<code>black_scholes.py</code>	analytical pricing
<code>dataset.py</code>	synthetic data generation
<code>model.py</code>	neural network
<code>training.py</code>	training and prediction
<code>monte_carlo.py</code>	simulation benchmark
<code>svr.py</code>	classical ML baseline

Takeaway

Python orchestrates optimized numerical backends; the heavy computations are not pure Python loops.

Experiment Pipeline

The main experiment follows a deterministic workflow:

1. generate synthetic contracts;
2. compute analytical prices;
3. split and scale the data;
4. train the neural network;
5. evaluate against C_{BS} ;
6. save metrics, plots, configuration, and artifacts.

Takeaway

The experiment is reproducible from the command line, not tied to an interactive notebook.

Final Experiment Configuration

The final neural-network run uses:

Setting	Value
Dataset size	100,000 contracts
Feature set	base variables + moneyiness
Activation	SiLU
Maximum epochs	200
Batch size	1024
Monte Carlo paths	50,000

Model Selection Experiments

Intermediate experiments justify the final configuration.

Experiment	MAE	RMSE	MAPE
Base + ReLU	0.17269	0.22973	1.8687%
Moneyiness + ReLU	0.15417	0.20930	1.6670%
Base + SiLU	0.10775	0.15817	1.2488%
Moneyiness + SiLU	0.10254	0.15169	1.3213%

Takeaway

The combined moneyiness + SiLU setup gives the strongest MAE/RMSE candidate in the intermediate experiments.

Runtime Benchmark

Runtime is measured after training, for 1,000 pricing queries.

Method	Time	Options/s
Black-Scholes formula	0.00091 s	1,101,095
Neural inference	0.03190 s	31,351
Monte Carlo, 20,000 paths	1.28235 s	780

Takeaway

The analytical formula is fastest, but neural inference is about 40 times faster than Monte Carlo in this benchmark.

SVR Baseline

Support Vector Regression is included as a classical machine-learning reference.

Metric	Mean	Std
MAE	0.15088	0.00732
RMSE	0.23100	0.00592
R^2	0.999905	0.000005
MAPE	1.8253%	0.2422%

The benchmark is intentionally reduced to 5,000 samples over three seeds because RBF kernel methods scale less favorably.

Final Neural Network Results

Metric	Value
MAE	0.04290
RMSE	0.06208
R^2	0.999993
MAPE, prices > 1	0.4803%

Takeaway

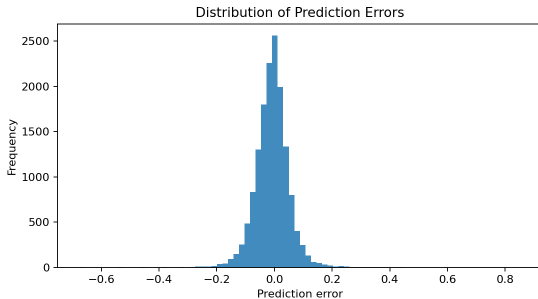
The final model approximates the Black-Scholes pricing function with very low error on held-out synthetic contracts.

Predicted vs Analytical Prices



Predictions are concentrated near the identity line, consistently with $R^2 \approx 1$.

Error Diagnostics



The error distribution checks whether low aggregate metrics hide large systematic deviations.

Noisy Target Robustness

Training labels are perturbed, while evaluation remains against clean Black-Scholes prices.

Noise level	NN MAE	SVR MAE
0%	0.10254	0.15088
1%	0.11084	0.19284
5%	0.17827	0.56185

Takeaway

Increasing label noise worsens performance, but the neural network remains reasonably stable under mild perturbations.

Interpretation and Limits

The results support a clear but conditional conclusion.

- ▶ Strong accuracy holds inside the sampled Black-Scholes domain.
- ▶ The model is a surrogate, not a replacement for the analytical formula.
- ▶ Synthetic data are appropriate for this research question.
- ▶ Real market data would define a different problem.
- ▶ No-arbitrage constraints are not explicitly enforced.

Final Takeaway

Answer

A feed-forward neural network can accurately approximate European call prices generated by the Black-Scholes formula in a controlled synthetic setting.

Future work:

- ▶ Greeks by automatic differentiation;
- ▶ stochastic-volatility models;
- ▶ path-dependent or American options;
- ▶ real market data as a separate research question.